1. **Which sorting algorithm exhibits a worst-case time complexity of O(n^2)?**
   c) Selection sort
2. **Which algorithm is suitable for finding the roots of a polynomial equation?**
   c) Newton-Raphson method
3. **Which sorting algorithm works by repeatedly stepping through the list, comparing each pair of adjacent elements, and swapping them if they are in the wrong order?**
   a) Bubble sort
4. **Which sorting algorithm is most suitable for small data sets or nearly sorted arrays due to its simplicity and efficiency?**
   b) Insertion sort
5. **Which sorting algorithm has a space complexity of O(1), making it efficient in terms of memory usage?**
   c) Selection sort
6. **Describe the process of the Bubble Sort algorithm and its main steps in sorting a list of elements.**

   Definition:

   Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process continues until the list is sorted.

   Main Steps of the Bubble Sort Algorithm:

   1. Start from the beginning of the array.
   2. Compare adjacent elements: If the current element is greater than the next one, swap them.
   3. Repeat step 2 for each pair until the end of the array, causing the largest unsorted element to "bubble up" to its correct position.
   4. Repeat the entire process for the remaining unsorted portion of the array.
   5. Stop when no swaps are needed in a complete pass — the array is now sorted.

   Example:

   Sort the array: [5, 3, 8, 4]

   - Pass 1:
     - Compare 5 & 3 → swap → [3, 5, 8, 4]
     - Compare 5 & 8 → no swap
     - Compare 8 & 4 → swap → [3, 5, 4, 8]
   - Pass 2:
     - Compare 3 & 5 → no swap
     - Compare 5 & 4 → swap → [3, 4, 5, 8]
     - Compare 5 & 8 → no swap
   - Pass 3:
     - Compare 3 & 4 → no swap

- Compare 4 & 5 → no swap → No swaps → sorted

Time and Space Complexity:

- Worst-case time: $O(n^2)$
- Best-case time (sorted list): $O(n)$
- Space complexity: $O(1)$ – in-place

Write the code for bubble sort.

```
#include<stdio.h>
void print(int arr[], int n){
    for(int i=0;i<n;i++){
        printf("%d", arr[i]);
    }
}

void sort(int arr[], int n){
    int temp;
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
           if(arr[j]<arr[i]){
            temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
           }
        }
    }
}
int main(void){
    int n;
    printf("Enter the no of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements into the array: \n");
    for(int i=0;i<n;i++){
        scanf("%d", &arr[i]);
    }
    printf("Original array: \n");
    print(arr, n);
    sort(arr,n);
    printf("\nAfter sorting array elements are: \n");
    print(arr, n);
    return 0;
}
```

7. Describe the process of the Selection Sort algorithm and its main steps in sorting a list of elements.

Definition:

Selection Sort is a simple comparison-based sorting algorithm. It repeatedly selects the smallest (or largest, depending on order) element from the unsorted portion of the list and places it at the correct position.

Main Steps:

1. **Start from the first element** of the array.
2. **Find the smallest element** in the remaining unsorted portion of the array.
3. **Swap** it with the first unsorted element.
4. **Move the boundary** of the sorted and unsorted parts one step forward.
5. **Repeat** the process until all elements are sorted.

Example:

For an array: [64, 25, 12, 22, 11]

- First pass: Find minimum (11), swap with 64 → [11, 25, 12, 22, 64]
- Second pass: Find minimum (12), swap with 25 → [11, 12, 25, 22, 64]
- Continue until the list is sorted.

Time Complexity:

- Best, Worst, Average: **$O(n^2)$**

Space Complexity:

- **$O(1)$** (In-place sorting)

Write the code for selection sort.

```
#include<stdio.h>
void print(int arr[], int n){
    for(int i=0;i<n;i++){
        printf("%d", arr[i]);
    }
}

void sort(int arr[], int n){
    int temp, small;
    for(int i=0;i<n-1;i++){
        small=i;
        for(int j=i+1;j<n;j++){
           if(arr[j]<arr[i]){
            small =j;
            temp=arr[small];
            arr[small]=arr[i];
            arr[i]=temp;
           }
        }
    }
}
int main(void){
    int n;
    printf("Enter the no of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements into the array: \n");
```

```
    for(int i=0;i<n;i++){
        scanf("%d", &arr[i]);
    }
    printf("Original array: \n");
    print(arr, n);
    sort(arr,n);
    printf("\nAfter sorting array elements are: \n");
    print(arr, n);
    return 0;
}
```

8. **Describe the process of the Insertion Sort algorithm and its main steps in sorting a list of elements.**

Definition:

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time by inserting each element into its proper position.

Main Steps:

1. Start from the **second element** (index 1), assuming the first element is already sorted.
2. **Compare** the current element with elements in the sorted part of the array (to its left).
3. **Shift** all elements that are greater than the current element one position to the right.
4. **Insert** the current element into its correct position.
5. **Repeat** the process for all elements until the entire list is sorted.

Example:

For an array: [5, 3, 4, 1, 2]

- First pass: Insert 3 → [3, 5, 4, 1, 2]
- Second pass: Insert 4 → [3, 4, 5, 1, 2]
- Third pass: Insert 1 → [1, 3, 4, 5, 2]
- Fourth pass: Insert 2 → [1, 2, 3, 4, 5]

Time Complexity:

- **Best Case (Already Sorted): O(n)**
- **Worst & Average Case: $O(n^2)$**

Space Complexity:

- **O(1)** (In-place sorting)

Write the code for selection sort.

```
#include<stdio.h>
void print(int arr[], int n){
    for(int i=0;i<n;i++){
```

```
        printf("%d ", arr[i]);
    }
}

void sort(int arr[], int n){
    int temp,j;
    for(int i=1;i<n;i++){
        temp=arr[i];
        j=i-1;
        while(j>=0 && temp<=arr[j]){
            arr[j+1]=arr[j];
            j=j-1;
        }
        arr[j+1]=temp;
    }
}
int main(void){
    int n;
    printf("Enter the no of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements into the array: \n");
    for(int i=0;i<n;i++){
        scanf("%d", &arr[i]);
    }
    printf("Original array: \n");
    print(arr, n);
    sort(arr,n);
    printf("\nAfter sorting array elements are: \n");
    print(arr, n);
    return 0;
}
```

Quick Sort Code.

```
#include<stdio.h>
int i;
void swap(int *a, int *b) {
    int temp=*a;
    *a=*b;
    *b=temp;
}

int partition(int a[], int lb, int ub) {
    int pivot=a[lb], start=lb, end=ub, temp;
    while(start<end){
        while(a[start]<=pivot)
            start++;
        while(a[end]>pivot)
            end--;
        if(start<end)
            swap(&a[start], &a[end]);
    }
    Swap (&a[lb], &a[end]);
```

```
    return end;
}


void qsort(int a[],int lb, int ub) {
    int q;
    if(lb<ub){
        q=partition(a,lb,ub);
        qsort(a,lb,q-1);
        qsort(a,q+1,ub);
    }
}

void printdata (int a[],int n) {
    for(i=0;i<n;i++)
    printf("\t%d",a[i]);
}

void main() {
    int arr[10],n;
    printf("Enter the size of the list: ");
    scanf("%d",&n);
    printf("\nEnter the unsorted list: ");
    for(i=0;i<n;i++)
    scanf("%d",&arr[i]);
    printf("\nThe unsorted list is: ");
    printdata(arr,n);
    qsort(arr,0,n-1);
    printf("\n\n\nThe sorted list is: ");
    printdata(arr,n);
}
```

9. **Describe the process of the Binary Search algorithm and its main steps in finding an element from a list of elements**.

Definition:
Binary Search is an efficient searching algorithm used to find the position of a target element in a **sorted list** by repeatedly dividing the search range in half.

Main Steps:
1. **Initialize two pointers**:
   low = 0 (start of the list)
   high = n - 1 (end of the list)
2. **Repeat until** low is less than or equal to high:
   - Calculate middle index: mid = (low + high) / 2
   - Compare the middle element arr[mid] with the target:
     - If arr[mid] == target: Element is found at index mid
     - If target < arr[mid]: Search in the left half → high = mid - 1
     - If target > arr[mid]: Search in the right half → low = mid + 1
3. If low > high, the element is **not found** in the list.

Example:
Given a sorted array: [10, 20, 30, 40, 50] and target = 30

- low = 0, high = 4, mid = 2 → arr[2] = 30 → **Element found**

Time Complexity:

- **Best Case:** $O(1)$
- **Average/Worst Case:** $O(\log n)$

Space Complexity:

- $O(1)$ (iterative), $O(\log n)$ (recursive)

Code:

```
#include <stdio.h>

int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == target)
            return mid;  // Element found
        else if (arr[mid] < target)
            low = mid + 1;  // Search right half
        else
            high = mid - 1; // Search left half
    }
    return -1;  // Element not found
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 30;
    int result = binarySearch(arr, n, target);
    if (result != -1)
        printf("Element found at index %d\n", result);
    else
        printf("Element not found in the array\n");
    return 0;
}
```

Describe the process of the Linear Search algorithm and its main steps in finding an element from a list of elements.

Definition:
Linear Search is a simple searching algorithm used to find a target element in a list by checking each element one by one from the beginning to the end.

Main Steps:

1. **Start from the first element** of the list.

2. **Compare** each element with the target value.
3. If a match is found, **return the index** (or position) of the element.
4. If the end of the list is reached without finding the element, **report that it is not present**.

Example:

Given array: [15, 28, 39, 42, 56], target = 42

- Step 1: Compare 15 → no match
- Step 2: Compare 28 → no match
- Step 3: Compare 39 → no match
- Step 4: Compare 42 → **match found at index 3**

Time Complexity:

- **Best Case:** O(1) (if element is at the beginning)
- **Worst/Average Case:** O(n) (if element is at the end or not present)

Space Complexity:

- ***O*(1)**

Code:

```
#include <stdio.h>

int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target)
            return i;  // Element found at index i
    }
    return -1;  // Element not found
}

int main() {
    int arr[] = {15, 28, 39, 42, 56};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 42;

    int result = linearSearch(arr, n, target);

    if (result != -1)
        printf("Element found at index %d\n", result);
    else
        printf("Element not found in the array\n");

    return 0;
}
```

10. Differentiate between linear search and binary search algorithms in terms of their approach to searching and their time complexity.

Feature	Linear Search	Binary Search
Approach	Checks each element one by one	Repeatedly divides the search range in half

Requirement	Works on unsorted or sorted lists	Works only on sorted lists
Time Complexity	Best: $O(1)$ (first element) Worst: $O(n)$	Best: $O(1)$ (middle element) Worst: $O(\log n)$
Efficiency	Less efficient for large lists	More efficient for large, sorted lists
Implementation	Simple and easy to implement	Slightly more complex, needs sorting if not already done